CSA0419 - OPERATING SYSTEMS

ASSIGNMENT 2

1. Understanding the assignment

This assignment focuses on how an operating system manages multiple processes, distributes CPU time, allows processes to communicate, prevents data conflicts and handles deadlocks.

There are six main questions, each based on a real-world computing scenario.

Question	Topic	Marks
Q1	Processes and Process States	6
Q2	CPU Scheduling	8
Q3	Threads	6
Q4	Inter-Process Communication	6
Q5	Synchronization, Semaphores and Monitors	11
Q6	Deadlock Handling and Banker's Algorithm	13
Total		50

Important instruction: Every question is at K5 – Evaluate level. This means your answers must include explanations, comparisons, advantages, disadvantages, calculations, evidence and justified decisions.

Q1 – Processes and Process States (6 Marks)

Scenario: Online Examination Server

An online examination server manages 2,000 students at the same time. Each student has a separate examination session. The server also performs automatic answer saving and video monitoring.

When many students log in simultaneously, the server becomes slow.

A. What is a process?

A process is a program that is currently being executed by the CPU.

For example:

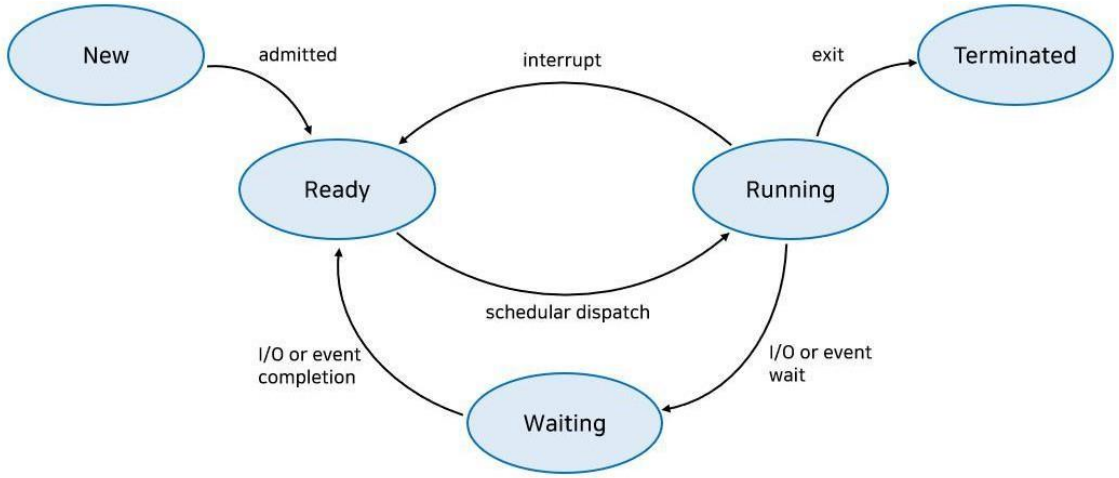
- Student 1 examination session is one process.
- Student 2 examination session is another process.
- Automatic answer saving can run as another process.
- Proctoring video processing can run as another process.

B. Process states

A process passes through five main states.

State	Explanation
New	Process is being created
Ready	Waiting for CPU allocation
Running	Currently executing on CPU
Waiting	Waiting for I/O or an event
Terminated	Execution has completed

Process-state diagram



Explanation of transitions:

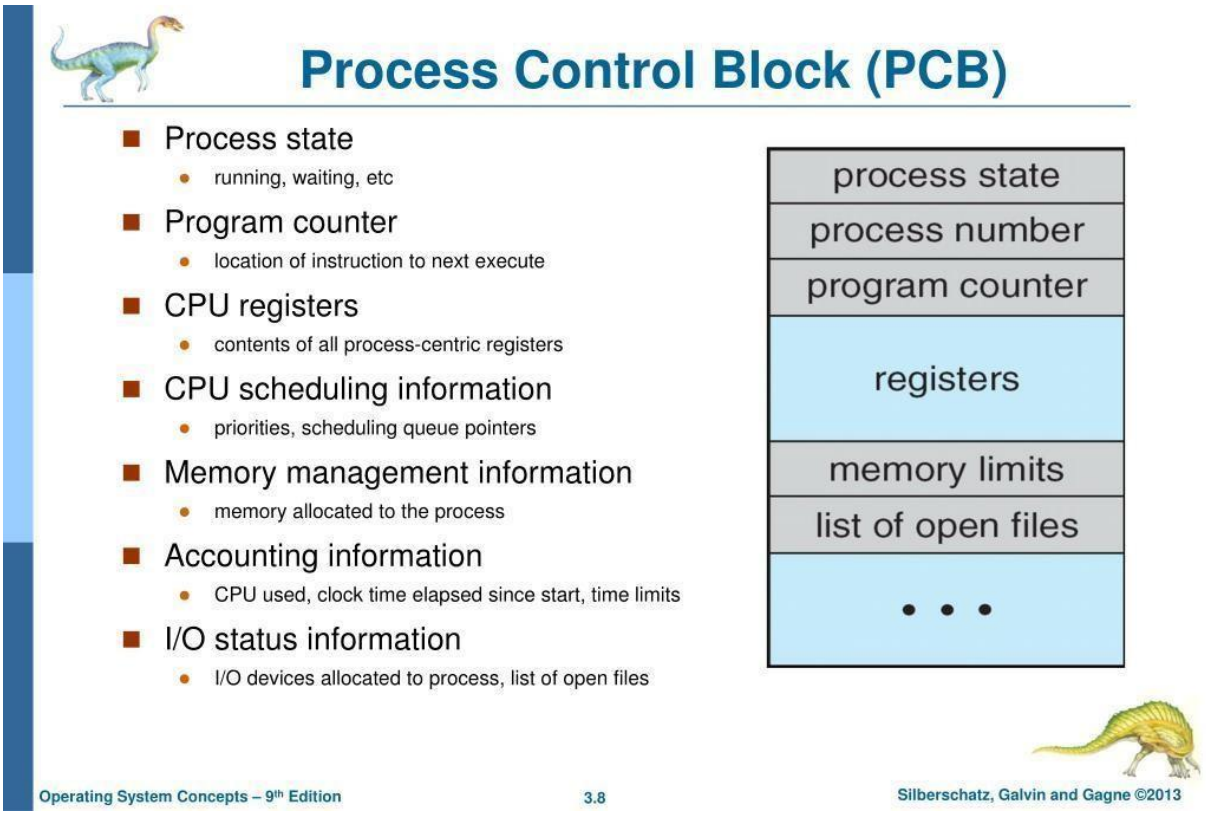
1. New → Ready: The OS admits the process.

2. Ready → Running: CPU scheduler selects the process.
3. Running → Waiting: Process requests I/O.
4. Waiting → Ready: I/O operation completes.
5. Running → Ready: CPU time quantum expires or process is preempted.
6. Running → Terminated: Process completes execution.

C. Process Control Block (PCB)

PCB is a data structure maintained by the operating system to store information about a process.

PCB component	**Purpose**
Process ID	Unique process identification
Process State	Current state
Program Counter	Next instruction address
CPU Registers	Stores CPU register values
CPU Scheduling Information	Priority and scheduling details
Memory Information	Memory allocation details
I/O Information	Open files and I/O devices
Accounting Information	CPU usage and execution time



slideserve.com

D. Create child processes using fork()

The fork() system call creates a child process from a parent process.

C

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
   int pid = fork();
   if (pid == 0)
      printf("Child process running\n");
   else if (pid > 0) {
      printf("Parent process running\n");
      wait(NULL);
   }
```

```
    else
        printf("Process creation failed\n");
    return 0;
}
```

Compile and execute on Linux:

Bash

```
gcc process.c -o process
./process
```

E. Observe processes using Linux commands

Bash

```
ps -ef
```

Displays running processes.

Bash

```
ps -o pid,ppid,state,comm
```

Displays process IDs, parent IDs and states.

Bash

```
ls /proc
```

Displays process information maintained by Linux.

F. Evaluation of 2,000 processes

Running 2,000 processes can create performance problems.

- Context switching consumes CPU time.
- Every process requires its own virtual address space.
- Memory usage increases.
- Processes waiting for disk or network I/O occupy resources.
- Too many runnable processes increase scheduling overhead.

G. Final design decision

Design	Advantages	Disadvantages
One process per student	Strong isolation	High memory overhead
One thread per student	Lower memory overhead	Shared-memory risks
Fixed worker process pool	Controlled resource usage	Requires task distribution

Recommendation: A fixed worker pool with threads is a suitable design for the examination server. It can handle many student sessions without creating thousands of separate processes.

H. Alternative concept – One thread per student

If every student session uses a thread instead of a separate process:

- Threads share the same process address space.
- Each thread has its own stack and execution state.
- Context switching between threads is generally cheaper.
- Memory usage is lower than creating a complete process for every session.
- A faulty thread may affect the entire server process.

Complexity comparison:

- Creating N processes requires O(N) process-creation operations.
- Creating N threads requires O(N) thread-creation operations.
- Process memory overhead is generally higher than thread memory overhead.

Conclusion: Threads or a controlled worker pool are better for managing large numbers of student sessions.

Q2 – CPU Scheduling (8 Marks)

8 Marks

Scenario: Cloud Video-Encoding Service

A video service receives two types of jobs:

- Short jobs: Thumbnail generation (2 seconds).
- Long jobs: Full-film encoding (20 minutes).

Customers complain that short jobs wait too long because long jobs occupy the CPU.

A. What is CPU scheduling?

CPU scheduling is the process of selecting a process from the ready queue and allocating CPU time to it.

The main objective is to improve CPU utilization, reduce waiting time and provide faster response.

B. Important CPU scheduling algorithms

Algorithm	Explanation	Advantage	Disadvantage
FCFS	Executes processes in arrival order	Simple	Long jobs delay short jobs
SJF	Executes the shortest job first	Low average waiting time	Starvation possible
SRTF	Executes the process with the shortest remaining time	Better response for short jobs	More context switches
Priority	Executes the highest-priority process first	Important jobs get preference	Low-priority starvation
Round Robin	Gives each process a fixed time quantum	Fair CPU allocation	Performance depends on quantum

C. Important scheduling formulas

1. Turnaround Time (TAT)

$$TAT = Completion\ Time - Arrival\ Time$$

2. Waiting Time (WT)

$$WT = Turnaround\ Time - Burst\ Time$$

3. Response Time (RT)

$$RT = First\ CPU\ Start\ Time - Arrival\ Time$$

4. Average Waiting Time

$$AWT = \frac{\sum WT}{n}$$

5. Average Turnaround Time

$$ATAT = \frac{\sum T\,AT}{n}$$

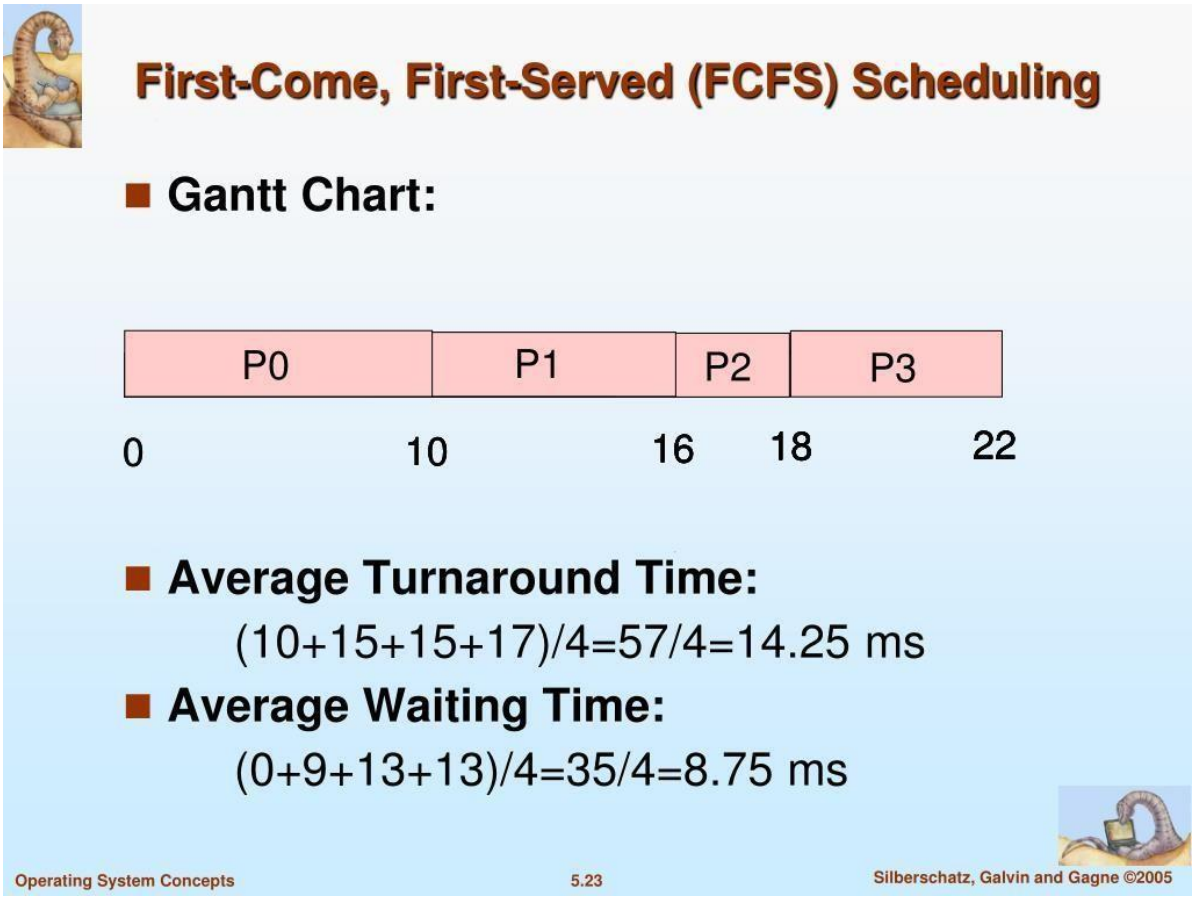
D. Given process details

The assignment's Round Robin program provides the following values.

Process	Arrival time	Burst time
P1	0	8
P2	1	4
P3	2	9
P4	3	5

E. FCFS scheduling

FCFS executes processes in their arrival order.



Gantt chart:

Plain text

0 8 12 21 26

| P1 | P2 | P3 | P4 |

Process	**Completion**	**TAT**	**WT**
P1	8	8	0
P2	12	11	7
P3	21	19	10
P4	26	23	18

$$AWT = \frac{0 + 7 + 10 + 18}{4} = 8.75$$

$$ATAT = \frac{8 + 11 + 19 + 23}{4} = 15.25$$

F. SJF scheduling

SJF selects the process with the shortest burst time among the available processes.

Plain text

0 8 12 17 26

| P1 | P2 | P4 | P3 |

Process	**Completion**	**TAT**	**WT**
P1	8	8	0
P2	12	11	7
P3	26	24	15
P4	17	14	9

$$AWT = \frac{0 + 7 + 15 + 9}{4} = 7.75$$

$$ATAT = \frac{8 + 11 + 24 + 14}{4} = 14.25$$

G. SRTF scheduling

SRTF is the preemptive version of SJF.

It selects the process with the shortest remaining execution time.

Plain text

0 1 5 10 17 26

|P1| P2 | P4 | P1 | P3 |

Process	**Completion**	**TAT**	**WT**
P1	17	17	9
P2	5	4	0
P3	26	24	15
P4	10	7	2

$$AWT = \frac{9 + 0 + 15 + 2}{4} = 6.5$$

$$ATAT = \frac{17 + 4 + 24 + 7}{4} = 13$$

H. Priority scheduling

Priority scheduling executes the highest-priority process first.

For this calculation, the assignment does not provide priority values. Therefore, you must clearly state the priority values you choose before preparing its Gantt chart and calculating its averages.

For example, if P1 has the highest priority, it will be selected before lower-priority jobs when scheduling permits.

I. Round Robin scheduling

Round Robin gives every process a fixed CPU time called a time quantum.

The given program uses:

$$Quantum = 3$$

Gantt chart:

Plain text

0-3 P1

3-6 P2

6-9 P3

9-12 P4

12-15 P1

15-16 P2

16-19 P3

19-21 P4

21-23 P1

23-26 P3

Process	**TAT**	**WT**
P1	23	15
P2	15	11
P3	24	15
P4	18	13

$$AWT = \frac{15 + 11 + 15 + 13}{4} = 13.5$$

$$ATAT = \frac{23 + 15 + 24 + 18}{4} = 20$$

J. Complete the Round Robin program – TODO 1

Replace the TODO section with the following code.

C

```
int run = (left[p] < quantum) ? left[p] : quantum;

int start = time;
time += run;

printf(" %s(%d-%d)", name[p], start, time);

left[p] -= run;

/* Add newly arrived processes */
for (int i = 0; i < N; i++) {
    if (!added[i] && arrival[i] <= time) {
        queue[tail++] = i;
        added[i] = 1;
    }
}

/* Reinsert unfinished process */
if (left[p] > 0) {
    queue[tail++] = p;
} else {
    finish[p] = time;
    done++;
}
```

Insert this inside the while (done < N) loop, after retrieving p from the queue.

Compile:

Bash

```
gcc -std=c11 -Wall -Wextra rr.c -o rr
```

Run:

Bash

```
./rr
```

K. Effect of different time quantum values

The assignment asks you to test quantum values 1, 3, 5 and 20.

Quantum	Expected behaviour
1	More context switches and frequent process rotation
3	Balanced sharing for the given example
5	Fewer context switches
20	Behaves similarly to FCFS for these jobs

L. Starvation and ageing

Starvation: A process waits for a very long time because other processes keep receiving CPU time.

Ageing: Gradually increases the priority of waiting processes to prevent starvation.

M. Multilevel Feedback Queue (MLFQ)

MLFQ uses multiple ready queues with different priorities and time quanta.

- New jobs enter a high-priority queue.
- Short jobs finish quickly.
- Jobs using their full time quantum move to lower-priority queues.
- Longer jobs receive CPU time at lower levels.
- Ageing can prevent starvation.

N. Alternative concept and complexity analysis

Using MLFQ instead of one Round Robin queue:

- Short jobs can receive faster service.
- Long jobs cannot continuously dominate the CPU.
- Multiple queues require additional scheduling management.
- A queue-based scheduling decision can be O(1) with suitable implementation.
- SJF selection can require O(n) scanning of the ready processes.

Final recommendation: MLFQ is suitable for the cloud video-encoding service because it gives preference to short interactive jobs while still allowing long encoding jobs to execute.

Q3 – Threads (6 Marks)

Scenario: Traffic-Camera Number-Plate Reader

A city has 64 traffic cameras. Each camera continuously captures images and sends them for number-plate recognition.

Processing cameras one after another causes delays during peak hours.

A. What is a thread?

A thread is the smallest unit of CPU execution within a process.

Threads allow multiple tasks to run concurrently within one application.

For example:

- Thread 1 processes Camera 1.
- Thread 2 processes Camera 2.
- Thread 3 processes Camera 3.
- Other threads process the remaining cameras.

B. Resources shared by threads

Shared resources	**Private resources**
Code section	Thread stack
Data section	Registers
Heap memory	Program counter

Open files	Thread ID
Process resources	Thread execution state

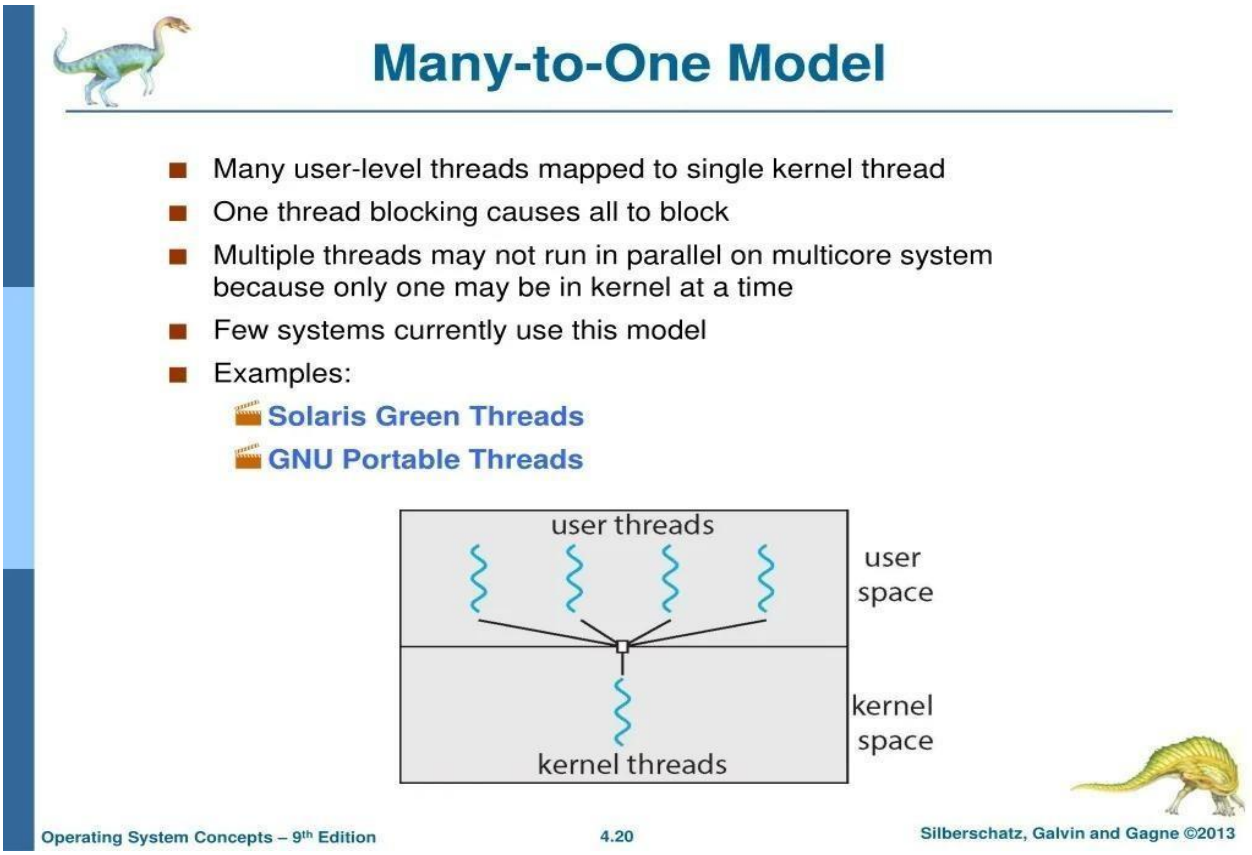
C. User-level vs kernel-level threads

User-level threads	**Kernel-level threads**
Managed by a user-level library	Managed by the OS kernel
Lower management overhead	Higher management overhead
Kernel may not see individual threads	Kernel schedules individual threads
A blocking operation can affect the process depending on the model	Other threads can continue running

D. Thread mapping models

1. Many-to-One

Many user threads are mapped to one kernel thread.



2. One-to-One

Each user thread maps to one kernel thread.

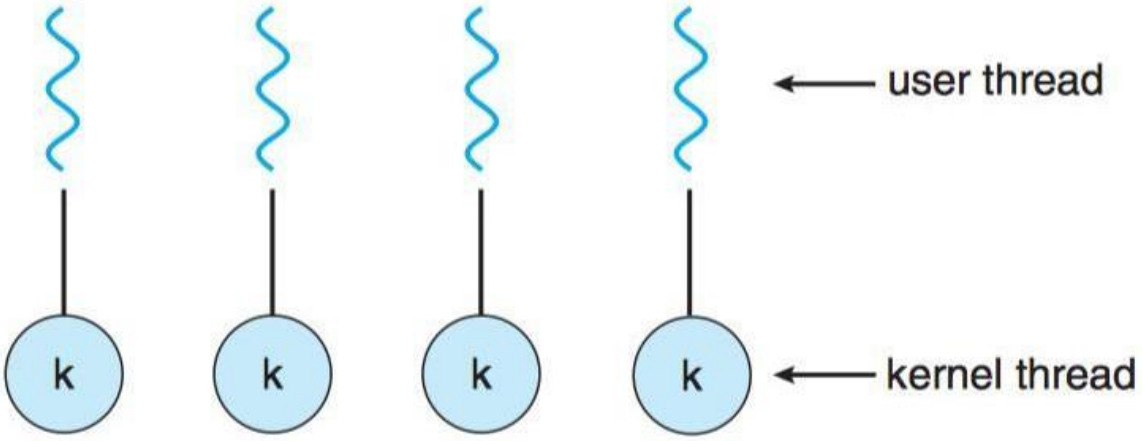
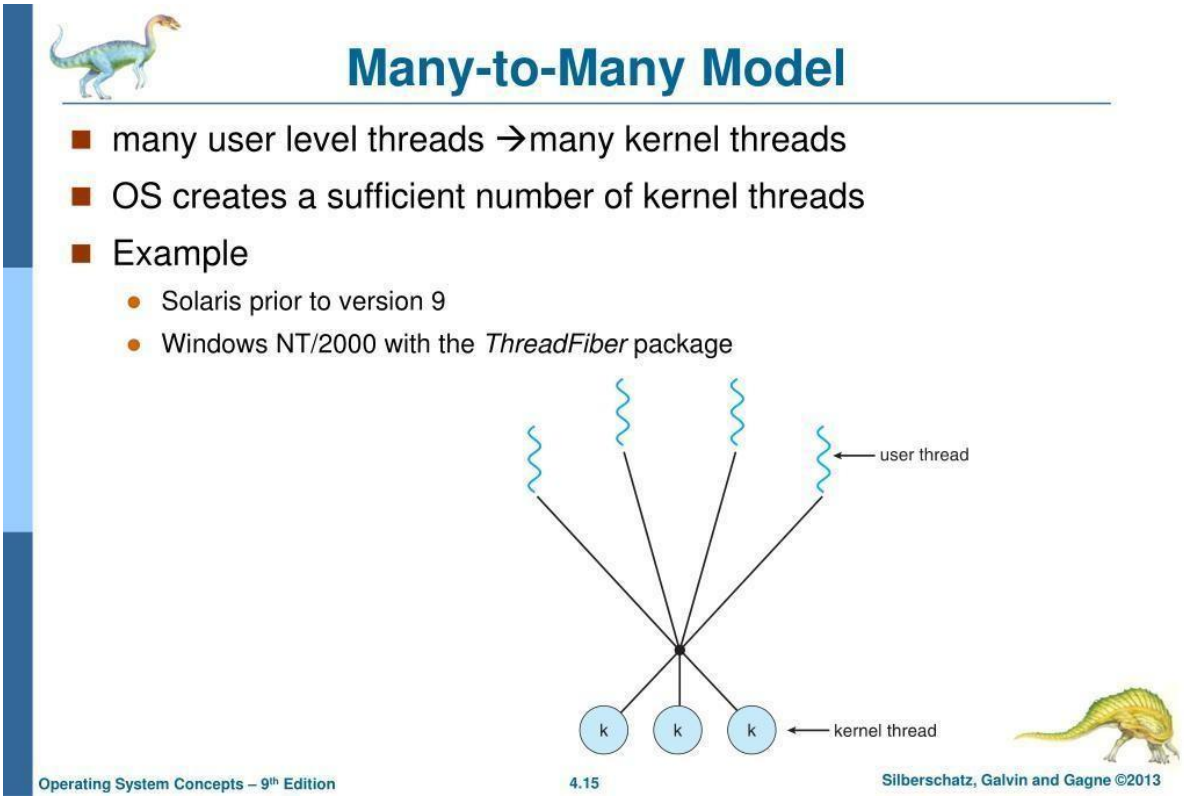


Figure 4.6 One-to-one model.

3. Many-to-Many

Many user threads are mapped to multiple kernel threads.



E. Simple POSIX thread program

This program creates multiple threads to process camera numbers.

C

```
#include <stdio.h>
#include <pthread.h>
void *camera(void *arg)
{
    int id = *(int *)arg;
    printf("Camera %d processed\n", id);
    return NULL;
}
int main()
{
    pthread_t t[4];
    int id[4];
    for (int i = 0; i < 4; i++) {
        id[i] = i + 1;
        pthread_create(&t[i], NULL, camera, &id[i]);
    }
    for (int i = 0; i < 4; i++)
        pthread_join(t[i], NULL);
    printf("All cameras processed\n");
    return 0;
}
```

Compile:

Bash

```
gcc -std=c11 -Wall -Wextra -pthread camera.c -o camera
```

Execute:

Bash

./camera

F. Compare different thread-pool sizes

The assignment requires testing one thread per camera and pools of 4, 8 and 16 threads.

Thread configuration	**Expected behaviour**
1 thread	Sequential processing
4 threads	Four frames processed concurrently
8 threads	Greater concurrency
16 threads	Higher concurrency, but more scheduling overhead
64 threads	One thread per camera

Measure actual execution time on your computer.

G. Speed-up calculation

$$Speedup = \frac{Sequential\ Execution\ Time}{Parallel\ Execution\ Time}$$

For example, if sequential execution takes 8 seconds and parallel execution takes 4 seconds:

$$Speedup = \frac{8}{4} = 2$$

This means the parallel implementation is twice as fast.

H. Why does speed-up stop increasing?

- Limited number of CPU cores.
- Thread creation overhead.
- Context switching.
- Shared resource contention.
- Synchronization overhead.
- Memory bandwidth limitations.

I. Shared-counter problem

If two threads update a shared counter without protection, a race condition may occur.

For example:

C

counter++;

This operation involves reading, modifying and writing the counter. Two threads may interfere with each other and cause lost updates.

A mutex or atomic operation can prevent this problem.

J. Alternative concept – Multiple processes instead of threads

Processes	**Threads**
Separate memory spaces	Shared process memory
Higher creation overhead	Lower creation overhead
IPC needed for data sharing	Shared variables can be used
Stronger isolation	Less isolation
Higher memory consumption	Lower memory consumption

Complexity:

- Creating N processes: O(N) operations.
- Creating N threads: O(N) operations.
- Combining process results requires IPC, while threads can share results through common memory with synchronization.

Final recommendation: A thread pool of 8 or 16 workers is a reasonable starting design for 64 cameras. The best size must be determined through actual performance measurements on the available CPU.

Q4 – Inter-Process Communication (6 Marks)

Scenario: Weather Station Data Pipeline

A weather station contains two processes:

1. Sensor reader – collects temperature and other readings.

2. Data analyser – calculates average values and maximum readings.

These processes must exchange information without losing data.

A. What is IPC?

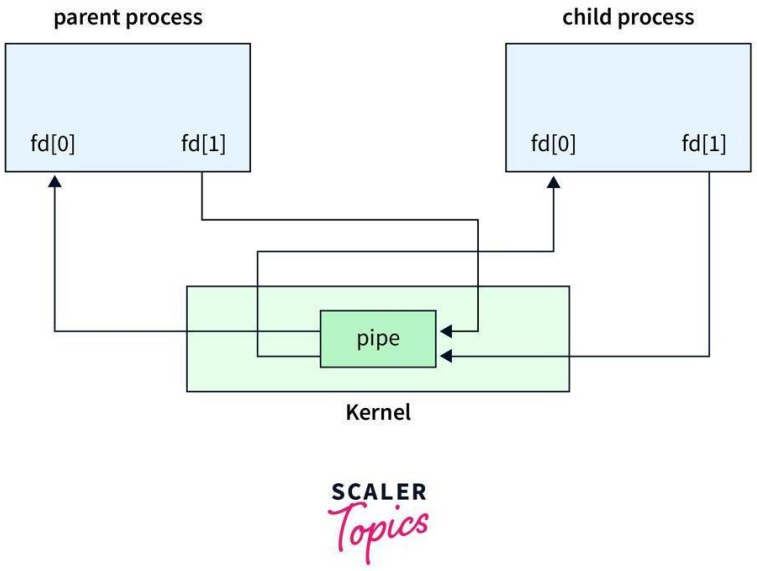
Inter-Process Communication (IPC) is a mechanism that allows two or more processes to exchange information.

B. IPC methods

Method	**Explanation**
Pipe	Transfers data between related processes
Shared memory	Allows processes to access a common memory region
Message queue	Transfers structured messages between processes

C. Pipe communication

A pipe provides a communication channel between two processes.



scaler.com

Working:

1. Parent creates a pipe.
2. Parent creates a child using fork().
3. Parent writes sensor readings into the pipe.
4. Child reads the readings.
5. Parent closes the writing end.
6. Child receives end-of-file and finishes reading.

D. Complete the pipe program – TODO 1

Replace the TODO section with:

C

```
while (read(fd[0], &r, sizeof(r)) > 0) {
    count++;
    sum += r
    if (count == 1 || r > max)
        max = r;
}
```

E. Complete program

C

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void)
{
    int fd[2];
    if (pipe(fd) == -1) {
        perror("pipe");
        return 1;
    }
```

```
pid_t pid = fork();
if (pid < 0) {
    perror("fork");
    return 1;
}
if (pid == 0) {
    close(fd[1]);
    int r, count = 0, sum = 0, max = 0;
    while (read(fd[0], &r, sizeof(r)) > 0) {
        count++;
        sum += r;
        if (count == 1 || r > max)
            max = r;
    }
    printf("analyser: %d readings, average %.1f, max %d\n",
           count, count ? (double)sum / count : 0.0, max);
    close(fd[0]);
    fflush(stdout);
    _exit(0);
}
close(fd[0]);
int readings[] = {31, 33, 36, 34, 41, 38};
for (int i = 0; i < 6; i++) {
    if (write(fd[1], &readings[i],
              sizeof(readings[i])) != sizeof(readings[i])) {
        perror("write");
        close(fd[1]);
        waitpid(pid, NULL, 0);
        return 1;
```

```
        }
    }
    close(fd[1]);
    waitpid(pid, NULL, 0);
    printf("reader: done\n");
    return 0;
}
```

Compile:

Bash

```
gcc -std=c11 -Wall -Wextra pipe.c -o pipe
```

Run:

Bash

```
./pipe
```

F. Expected output

Plain text

```
analyser: 6 readings, average 35.5, max 41
reader: done
```

G. How does the pipe identify the end of data?

When the parent closes the write end of the pipe and all other write descriptors are closed, the child receives 0 from read().

This indicates that no more data is available.

What happens if close(fd[1]) is removed?

The child continues waiting because the pipe still has an open writing end. It does not receive the end-of-file indication, so the reading loop does not finish.

H. Shared memory vs message passing

Shared memory	**Pipe / message passing**
Processes access common memory	Data is transferred through communication channels
Can avoid repeated data copying after setup	Pipe data is copied through kernel-managed buffers
Requires synchronization	Pipe manages basic transfer coordination
High performance for large data	Easier to implement

I. Alternative concept – Shared memory with semaphore

Instead of a pipe:

- Create a shared memory segment.
- Store sensor readings in the shared region.
- Use a semaphore to coordinate the reader and analyser.
- The analyser reads data directly from shared memory.

Complexity comparison:

For N readings:

- Pipe transfer requires O(N) data-transfer work and kernel-mediated copying.
- Shared memory requires O(N) data processing but avoids repeated pipe transfers.
- Shared memory requires additional synchronization to prevent reading incomplete or overwritten data.

J. Growth investigation

The assignment requires measuring transfer times for:

Data size	**Pipe time**	**Shared-memory time**
1 MB	Measure	Measure
10 MB	Measure	Measure

100 MB	Measure	Measure

Final recommendation: Shared memory with proper synchronization is suitable for large, high-speed data transfer. Pipes are simpler and suitable for basic communication between related processes.

Q5 – Synchronization: Critical Sections, Semaphores and Monitors (11 Marks)

11 Marks

Scenario: Railway Ticket Booking System

Thousands of booking agents are booking seats on the same trains simultaneously.

Sometimes the same seat is booked twice because multiple agents access and update the seat counter at the same time.

A. What is synchronization?

Synchronization is the process of coordinating multiple processes or threads to ensure that shared resources are accessed safely.

For example, when two agents try to book the same railway seat, synchronization ensures that only one agent can successfully reserve it.

B. Critical-section problem

A critical section is a part of a program where shared resources are accessed or modified.

Example:

C

seat_count--;

If multiple threads execute this statement simultaneously without protection, incorrect booking results may occur.

C. Three requirements of a critical-section solution

Requirement	**Explanation**
Mutual exclusion	Only one process can enter the critical section at a time
Progress	If no process is inside the critical section, an eligible waiting process should be allowed to enter without unnecessary delay

Bounded waiting	A process should not wait indefinitely to enter the critical section

D. Peterson's solution

Peterson's algorithm is a software-based synchronization method for two processes.

It uses:

- flag[2] – Indicates whether a process wants to enter the critical section.
- turn – Decides whose turn it is.

Example:

C

```
flag[i] = 1;

turn = j;

while (flag[j] && turn == j);

/* Critical section */

flag[i] = 0;
```

Advantages:

- Provides mutual exclusion for two processes.
- Provides progress.
- Provides bounded waiting under its assumptions.

Limitations:

- Designed for two processes.
- Depends on memory-ordering assumptions.
- May fail if used without suitable atomic operations and memory barriers on modern processors and compilers.

E. Semaphores

A semaphore is a synchronization variable used to control access to shared resources.

Two main operations are:

- wait() – Decreases the semaphore value or blocks the caller if the resource is unavailable.
- signal() – Increases the semaphore value and may wake a waiting process.

Types of semaphores

Binary semaphore	**Counting semaphore**
Has values 0 and 1	Can have multiple non-negative values
Controls one resource or access permission	Controls multiple identical resources
Used for mutual exclusion	Used for resource counting

F. Railway booking using a binary semaphore

Example:

C

```
sem_wait(&lock);
/* Critical section */
seat_count--;
sem_post(&lock);
```

This ensures that only one thread updates the shared seat counter at a time.

G. Classical synchronization problems

1. Producer–Consumer problem

- Producer generates data.
- Consumer processes data.
- A shared buffer stores items.
- Semaphores prevent buffer overflow and underflow.

2. Readers–Writers problem

- Multiple readers can access shared data simultaneously.
- A writer requires exclusive access.
- Synchronization prevents reading incomplete or modified data.

3. Dining Philosophers problem

- Philosophers share limited forks.
- Each philosopher needs two forks to eat.
- Improper resource allocation can cause deadlock.

- Semaphores can control access and prevent unsafe allocation.

H. Monitors and condition variables

A monitor is a high-level synchronization mechanism that combines shared data and operations into one protected structure.

Condition variables allow threads to wait until a particular condition becomes true.

Advantages:

- Provides structured synchronization.
- Helps avoid race conditions.
- Makes shared-resource management easier.
- Reduces the complexity of manually managing locks.

I. Bounded-buffer problem

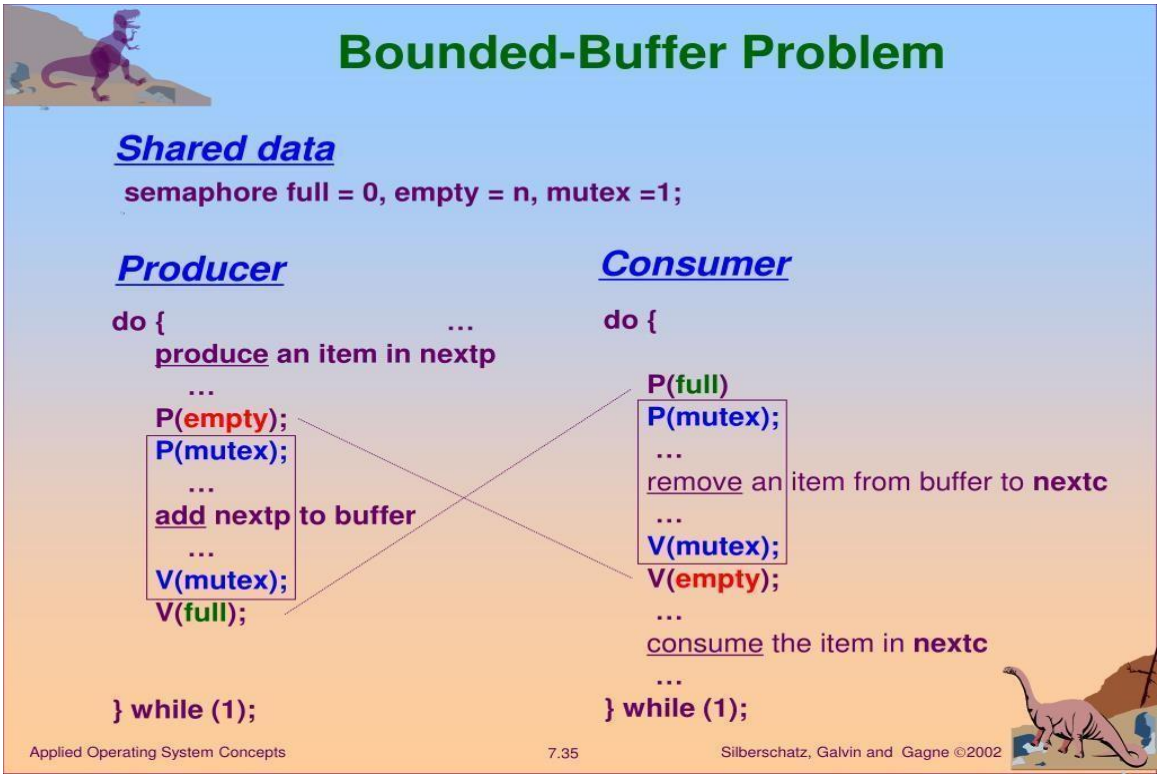
A bounded buffer has a fixed capacity.

In this assignment:

$$SIZE = 5$$

$$ITEMS = 20$$

The producer inserts 20 items, and the consumer removes them.



Three synchronization variables are used:

Variable	Purpose
empty	Counts available buffer spaces
full	Counts filled buffer spaces
lock	Protects shared buffer access

J. Complete the given semaphore program – TODO 1 and TODO 2

TODO 1 – Producer function

Replace the missing code inside the producer loop with:

C

```
sem_wait(&empty);
pthread_mutex_lock(&lock);
buffer[in] = item;
in = (in + 1) % SIZE;
pthread_mutex_unlock(&lock);
sem_post(&full);
```

Explanation:

1. Wait for an empty buffer position.
2. Lock the mutex.
3. Insert the item.
4. Update the circular buffer index.
5. Unlock the mutex.
6. Inform the consumer that an item is available.

TODO 2 – Consumer function

Replace the missing code inside the consumer loop with:

C

```
int item;
sem_wait(&full);
pthread_mutex_lock(&lock);
item = buffer[out];
out = (out + 1) % SIZE;
pthread_mutex_unlock(&lock);
sem_post(&empty);
consumed_sum += item;
```

Explanation:

1. Wait until an item is available.
2. Lock the mutex.
3. Remove the item.
4. Update the circular buffer index.
5. Unlock the mutex.
6. Inform the producer that a buffer position is empty.
7. Add the consumed item to the total.

K. Complete program

C

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#define SIZE 5
#define ITEMS 20
int buffer[SIZE], in = 0, out = 0;
sem_t empty, full;
```

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
long consumed_sum = 0;
void *producer(void *arg)
{
    (void)arg;
    for (int item = 1; item <= ITEMS; item++) {
        sem_wait(&empty);
        pthread_mutex_lock(&lock);
        buffer[in] = item;
        in = (in + 1) % SIZE;
        pthread_mutex_unlock(&lock);
        sem_post(&full);
    }
    return NULL;
}
void *consumer(void *arg)
{
    (void)arg;
    for (int k = 0; k < ITEMS; k++) {
        int item;
        sem_wait(&full);
        pthread_mutex_lock(&lock);
        item = buffer[out];
        out = (out + 1) % SIZE;
        pthread_mutex_unlock(&lock);
        sem_post(&empty);
        consumed_sum += item;
    }
```

```
    return NULL;
}
int main(void)
{
    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_t p, c;
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    printf("items consumed: %d, sum = %ld (expected %d)\n",
           ITEMS, consumed_sum, ITEMS * (ITEMS + 1) / 2);
    sem_destroy(&empty);
    sem_destroy(&full);
    return 0;
}
```

L. Compilation and execution

Bash

```
gcc -std=c11 -Wall -Wextra -pthread semaphore.c -o semaphore
```

Run:

Bash

```
./semaphore
```

M. Expected output

Plain text

```
items consumed: 20, sum = 210 (expected 210)
```

N. Run the program 100 times

Linux command:

Bash

for i in $(seq 1 100); do ./semaphore; done

Every successful execution should produce the expected sum of 210.

O. What happens if the mutex is removed?

- Multiple threads may access the buffer simultaneously.
- Two producers may write to the same position.
- Two consumers may read the same item.
- Data may be lost or duplicated.
- The final sum may become incorrect.

The mutex protects the shared buffer indices and contents. The semaphores control the number of occupied and empty positions.

P. Deployment decision – Locking strategies

The assignment asks you to compare three locking strategies.

Locking strategy	**Advantage**	**Disadvantage**
One global lock	Simple to implement	Low concurrency
One lock per train	Different trains can be booked simultaneously	Agents booking the same train must wait
One lock per coach	Greater concurrency	More complex seat and booking coordination

Recommendation: One lock per train is a good balance between performance and simplicity. It allows different trains to be booked simultaneously while preventing conflicting seat reservations within the same train.

You must measure actual throughput before finalizing this choice.

Q. Alternative concept – Atomic operations and spinlocks

Atomic operation: Performs an update as one indivisible operation.

Example:

C

atomic_fetch_add(&counter, 1);

Spinlock: A thread repeatedly checks whether a lock is available instead of sleeping.

Atomic operations	**Spinlocks**	**Semaphores and mutexes**
Suitable for simple counter updates	Suitable for very short critical sections	Suitable for longer critical sections
Low overhead	Can waste CPU while spinning	Can block waiting threads
Does not protect complex multi-step operations automatically	Provides exclusive access	Provides controlled access

Evaluation:

- Atomic operations are suitable for simple counters.
- Spinlocks may be useful for very short operations.
- Mutexes and semaphores are better for longer operations because they avoid continuously wasting CPU time.

R. Time and space complexity

For N producer or consumer operations:

- Total insertion/removal operations: O(N).
- Each circular-buffer insertion/removal: O(1).
- Buffer space: O(SIZE).

The synchronization overhead depends on contention and the number of participating threads.

Final recommendation: Semaphores combined with mutexes are suitable for railway booking because they provide controlled access, prevent duplicate bookings and maintain data consistency.

Q6 – Deadlock Handling and Banker's Algorithm (13 Marks)

13 Marks – Highest-weighted question

Scenario: Shared Laboratory Equipment Scheduler

A research centre has multiple jobs requiring shared resources:

- GPUs
- Software licences

- High-speed storage

Sometimes jobs wait for resources held by other jobs, and the entire cluster stops.

This situation is called a deadlock.

A. Characterise deadlock (2 Marks)

What is deadlock?

Deadlock is a situation where two or more processes wait indefinitely for resources held by one another.

Four necessary conditions for deadlock

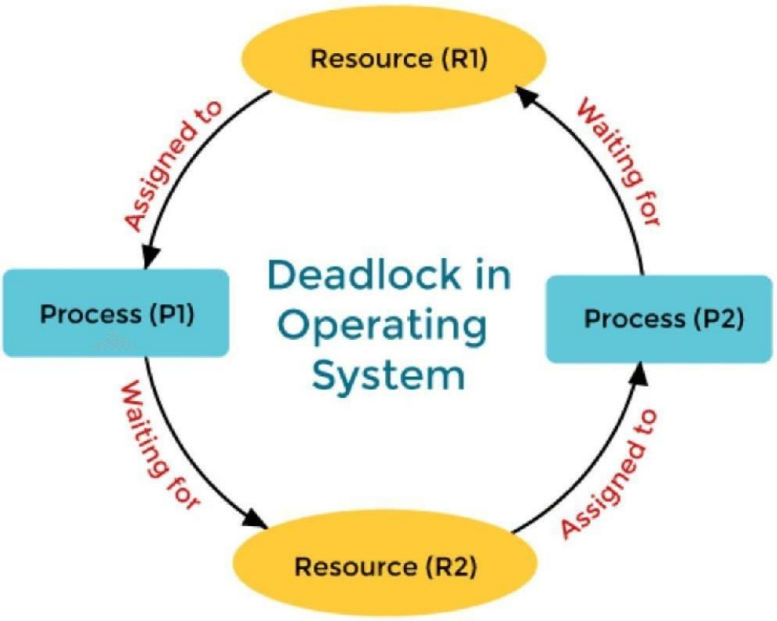
All four conditions must exist simultaneously for a deadlock to occur.

Condition	**Explanation**	**Example**
Mutual exclusion	A resource can be used by only one process at a time	One GPU assigned to one job
Hold and wait	A process holds one resource while waiting for another	Job holds GPU and waits for licence
No preemption	Resources cannot be forcibly taken away	GPU cannot be removed during execution
Circular wait	Processes form a circular chain of resource waiting	P1 waits for P2, and P2 waits for P1

Resource-allocation graph

A resource-allocation graph represents processes and resources using nodes and directed edges.

- Circle represents a process.
- Rectangle represents a resource.
- Process → Resource represents a request.
- Resource → Process represents an allocation.



almabetter.com

Example:

- P1 holds GPU and requests a licence.
- P2 holds a licence and requests GPU.
- P1 waits for P2.
- P2 waits for P1.

This creates a circular wait and deadlock.

B. Compare Deadlock-Handling Strategies (3 Marks)

There are four major deadlock-handling strategies.

1. Deadlock Prevention

Prevents deadlock by eliminating at least one of the four necessary conditions.

Examples:

- Request all required resources at once.
- Impose a fixed resource ordering.
- Allow resource preemption where appropriate.

Advantage: Deadlock can be prevented.

Disadvantage: Resource utilization may decrease.

2. Deadlock Avoidance

Checks whether granting a resource request will keep the system in a safe state.

Example: Banker's algorithm.

Advantage:

- Prevents the system from entering an unsafe state.
- Improves controlled resource allocation.

Disadvantage:

- Requires maximum resource demands in advance.
- Adds checking overhead.

3. Deadlock Detection and Recovery

Allows resource allocation and periodically checks whether deadlock has occurred.

Recovery methods:

- Terminate selected processes.
- Preempt resources where possible.
- Restart affected jobs.

Advantage: Resources can be allocated more flexibly.

Disadvantage: Work may be lost when processes are aborted.

4. Ignoring Deadlock (Ostrich Approach)

The system ignores deadlocks and assumes they are rare.

Advantage:

- Very low overhead.
- Simple implementation.

Disadvantage:

- Deadlocks can stop important jobs.
- Manual intervention may be required.

Comparison table

Strategy	Advantage	Disadvantage
Prevention	Avoids deadlock	May reduce utilization
Avoidance	Maintains safe allocation	Requires maximum demand information
Detection and recovery	Flexible allocation	Recovery can lose work
Ignoring	Simple and low overhead	Deadlock may stop the cluster

Recommendation: Deadlock avoidance using Banker's algorithm is suitable for the research centre when resource requirements can be estimated in advance.

C. Apply Banker's Algorithm (4 Marks)

1. What is Banker's algorithm?

Banker's algorithm is a deadlock-avoidance algorithm that checks whether granting a resource request keeps the system in a safe state.

It uses:

- Available
- Maximum
- Allocation
- Need

2. Given resource information

The assignment provides:

Available matrix

Resource	Available
A	3
B	3

C	2

Maximum matrix

Process	**A**	**B**	**C**
P0	7	5	3
P1	3	2	2
P2	9	0	2
P3	2	2	2
P4	4	3	3

Allocation matrix

Process	**A**	**B**	**C**
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2

3. Need matrix calculation

Formula:

$$Need = Maximum - Allocation$$

For P0:

$$Need = (7, 5, 3) - (0, 1, 0)$$

$$Need = (7, 4, 3)$$

Similarly, calculate for every process.

Final Need matrix

Process	A	B	C
P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1
P4	4	3	1

4. Safety algorithm

Steps:

1. Set Work = Available.
2. Set Finish = False for every process.
3. Find a process whose Need is less than or equal to Work.
4. If found, add its allocated resources to Work.
5. Mark that process as finished.
6. Repeat until all processes finish or no eligible process can be found.
7. If all processes finish, the system is safe.

5. Initial safe-sequence calculation

Initially:

$$Work = (3, 3, 2)$$

P1 can execute because:

$$(1, 2, 2) \leq (3, 3, 2)$$

After P1 finishes:

$$Work = (3, 3, 2) + (2, 0, 0)$$

$$Work = (5, 3, 2)$$

P3 can now execute:

$$(0, 1, 1) \leq (5, 3, 2)$$

After P3 finishes:

$$Work = (5, 3, 2) + (2, 1, 1)$$

$$Work = (7, 4, 3)$$

P4 can execute, followed by P0 and P2.

The final safe sequence is:

Safe sequence

$$P1 \rightarrow P3 \rightarrow P4 \rightarrow P0 \rightarrow P2$$

Therefore, the initial system state is safe.

6. Complete the Banker's Algorithm program – TODO 1

Replace the TODO section inside is_safe() with the following code.

C

```
int possible = 1;

for (int r = 0; r < R; r++) {
    if (maxm[p][r] - alloc[p][r] > work[r]) {
        possible = 0;
        break;
    }
}

if (possible) {
    for (int r = 0; r < R; r++)
```

```
            work[r] += alloc[p][r];

        finish[p] = 1;
        seq[count++] = p;
        found = 1;
    }
```

7. Complete program

C

```
#include <stdio.h>
#include <string.h>

#define P 5
#define R 3

int avail[R] = {3, 3, 2};

int maxm[P][R] = {
    {7, 5, 3},
    {3, 2, 2},
    {9, 0, 2},
    {2, 2, 2},
    {4, 3, 3}
};

int alloc[P][R] = {
    {0, 1, 0},
    {2, 0, 0},
    {3, 0, 2},
```

```
    {2, 1, 1},
    {0, 0, 2}
};

int is_safe(int seq[P])
{
    int work[R], finish[P] = {0}, count = 0;

    memcpy(work, avail, sizeof work);

    while (count < P) {
        int found = 0;

        for (int p = 0; p < P; p++) {
            if (finish[p])
                continue;

            int possible = 1;

            for (int r = 0; r < R; r++) {
                if (maxm[p][r] - alloc[p][r] > work[r]) {
                    possible = 0;
                    break;
                }
            }

            if (possible) {
                for (int r = 0; r < R; r++)
                    work[r] += alloc[p][r];
```

```
            finish[p] = 1;
            seq[count++] = p;
            found = 1;
        }
    }

    if (!found)
        return 0;
}

return 1;
}

void request(int p, const int req[R])
{
    for (int r = 0; r < R; r++) {
        if (req[r] > maxm[p][r] - alloc[p][r] ||
            req[r] > avail[r]) {
            printf("P%d request: must wait (not available)\n", p);
            return;
        }
    }

    for (int r = 0; r < R; r++) {
        avail[r] -= req[r];
        alloc[p][r] += req[r];
    }
```

```
    int seq[P];

    if (is_safe(seq)) {
        printf("P%d request: granted, safe sequence", p);

        for (int i = 0; i < P; i++)
            printf(" P%d", seq[i]);

        printf("\n");
    }
    else {
        for (int r = 0; r < R; r++) {
            avail[r] += req[r];
            alloc[p][r] -= req[r];
        }

        printf("P%d request: denied (unsafe)\n", p);
    }
}

int main(void)
{
    int seq[P];

    if (is_safe(seq)) {
        printf("initial state safe:");

        for (int i = 0; i < P; i++)
            printf(" P%d", seq[i]);
```

```
        printf("\n");
    }

    request(1, (int[R]){1, 0, 2});
    request(4, (int[R]){3, 3, 0});
    request(0, (int[R]){0, 2, 0});

    return 0;
}
```

8. Compile and execute

Save as banker.c.

Compile:

Bash

```
gcc -std=c11 -Wall -Wextra banker.c -o banker
```

Execute:

Bash

```
./banker
```

9. Expected output

Plain text

```
initial state safe: P1 P3 P4 P0 P2
P1 request: granted, safe sequence P1 P3 P4 P0 P2
P4 request: must wait (not available)
P0 request: denied (unsafe)
```

10. Evaluate each resource request

Request 1: P1 requests (1, 0, 2)

Available:

$$(3, 3, 2)$$

Requested:

$$(1, 0, 2)$$

Since the requested resources are available and the resulting state is safe, the request is granted.

Request 2: P4 requests (3, 3, 0)

After granting P1, available resources are:

$$(2, 3, 0)$$

P4 requests:

$$(3, 3, 0)$$

Since only two units of resource A are available, the request must wait.

Request 3: P0 requests (0, 2, 0)

After the P1 request is granted, available resources are:

$$(2, 3, 0)$$

After temporarily granting P0:

$$Available = (2, 1, 0)$$

Now check every unfinished process.

Process	**Need**	**Available Work**	**Can execute?**
P0	(7,2,3)	(2,1,0)	No
P1	(0,2,0)	(2,1,0)	No
P2	(6,0,0)	(2,1,0)	No
P3	(0,1,1)	(2,1,0)	No
P4	(4,3,1)	(2,1,0)	No

No process can complete.

Therefore, the system enters an unsafe state if the request is granted.

Request denied

Conclusion: P0's request must be denied because it does not allow the system to find a safe sequence.

D. Predict Before Implementing (1 Mark)

What does the question ask?

Predict how the running time of Banker's safety algorithm changes when the number of processes and resource types increases.

The assignment gives the expected complexity:

$$O(m \times n^2)$$

Where:

- n= Number of processes.
- m= Number of resource types.

Explanation

The algorithm checks each process and compares its resource requirements against the available resources.

As the number of processes increases, the number of comparisons also increases.

Processes (n)	**Resource types (m)**	**Relative complexity**
10	3	$3 \times 10^2 = 300$
100	3	$3 \times 100^2 = 30{,}000$
1000	3	$3 \times 1000^2 = 3{,}000{,}000$

This is an operation-count estimate based on the stated complexity, not measured execution time.

Conclusion: The running time increases approximately quadratically with the number of processes when the number of resource types remains constant.

E. Measure the Algorithm (1 Mark)

What does the question ask?

Execute the safety algorithm with 10, 100 and 1,000 processes and record the actual execution time.

Use the C clock() function to measure execution time.

Example:

C

```
#include <stdio.h>
#include <time.h>

int main(void)
{
    clock_t start, end;

    start = clock();

    /* Call your safety algorithm here */

    end = clock();
    double elapsed =
        (double)(end - start) / CLOCKS_PER_SEC;

    printf("Execution time: %f seconds\n", elapsed);

    return 0;
}
```

For your report, use this table:

Number of processes	**Resource types**	**Measured time**
10	3	Fill after execution
100	3	Fill after execution
1000	3	Fill after execution

Important: The original assignment code has fixed dimensions of 5 processes and 3 resources. To perform these additional measurements, extend the implementation to use configurable process counts and resource arrays. Use valid resource-allocation states for the benchmarks.

For reliable results:

- Run each test multiple times.
- Use the same computer and compiler.
- Record the average execution time.
- Compare the measured trend with the predicted $O(m \times n^2)$ growth.

F. Defend the Final Design (2 Marks)

What does the question ask?

Recommend the best deadlock-handling strategy for the research centre.

Recommended solution: Banker's algorithm

Reasons:

1. Prevents the system from entering an unsafe resource-allocation state.
2. Checks resource availability before granting requests.
3. Reduces the possibility of deadlock.
4. Helps manage shared GPUs, software licences and storage resources.

Limitations

- Maximum resource requirements must be known in advance.
- Safety checking introduces additional computational overhead.
- Frequent resource requests increase checking costs.
- It may delay some jobs while waiting for a safe allocation.

Final conclusion

Banker's algorithm is suitable when the research centre can estimate the maximum resource requirements of its jobs. It helps maintain a safe system state and improves resource allocation reliability. If resource requirements are unpredictable, detection and recovery may be more practical